Are Languages Represented in Disjoint Subspaces?

Probing the Multilingual Geometry of mBERT

Guillaume Wisniewski
guillaume.wisniewski@u-pariscite.fr

Academic year 2026–2027

Instructions — please read carefully — I mean really carefully

- **Deadline: 14 October 2026, 8:00 a.m.** Reports handed in late will not be marked.
- Perfect is the enemy of good: it is better to answer fewer questions properly than to rush through the whole assignment and do a poor job of everything.
- **What to send, by email:** a PDF whose name is your family name, prefixed with `m2_lab2_` (so, for me, and for me only: `m2_lab1_wisniewski.pdf`), together with a **`zip`** archive containing your source code.
- **For each question**, the report must give (a) a summary of the question and (b) your answer, written out (code, results, analysis). Keep it *as short as possible*: what is being marked is that you understood what you did and that you can interpret the result. There is no need for 300 lines of code: include only the essential or tricky part. As I said (with the greatest conviction) last year, most of the questions you will meet (in my labs and, more generally, in your life) can be answered in three lines of Python.
- **The report must open with a short text of 500 words, plus or minus 10 %** (that is, between 450 and 550 words — you can thank me for having done the maths for you), describing (a) what you did, (b) the difficulties you ran into, (c) the answers you arrived at and (d) what you learnt (most important part).
- Evaluation criteria: correctness of the answers, depth of analysis, code quality, originality, quality of reasoning, self-critical thinking, and quality of writing.
- **Sensible use of AI is allowed** (you will admire the artistic vagueness surrounding the word sensible), on two conditions: you document how you used it, and you explain how you made sure that what it produced was correct.

Context and objectives

Multilingual encoders such as mBERT are trained on the concatenation of monolingual corpora in about a hundred languages. They see no parallel data and receive no explicit signal telling

them which language a text is written in. Yet they transfer surprisingly well across languages: a classifier fine-tuned on English data often works reasonably well on Spanish or Hindi (Pires, Schlinger and Garrette 2019). How can a single vector space of dimension 768 host so many languages? Two caricatural answers are usually opposed.

- **Partition.** Each language occupies its own region of the space (a set of dimensions, a subspace), and the model largely processes languages separately.
- **Sharing.** Languages are mapped into a common space in which translations lie close to one another; the identity of the language is carried by a small, low-dimensional component.

The first answer is often summarised by the slogan 'language models use disjoint subspaces to represent different languages'. The aim of this practical is to test this hypothesis on a small multilingual model, with modest computing resources. You will quickly see that the hypothesis, *as stated*, cannot be tested: most of the work consists in turning it into precise predictions, each paired with a baseline telling you what 'high' and 'low' mean.

Structure

Part 1 *Warm-up (optional ☹).* How unequally does mBERT tokenise different languages? We test the idea that languages with little pre-training data are more fragmented.

Part 2 *Geometry.* Do the principal subspaces of different languages overlap? Is language identity more than a mean offset? Do directions mean the same thing across languages?

Part 3 *Neurons.* We re-implement the method of Tang et al. (2024) to find 'language-specific neurons' and test them causally.

You must do Part 2 or Part 3, and then the synthesis — although, personally, I think there is a lot to be learnt from doing both. Part 1 is optional (same remark): if you skip it, use Yoruba (`yor_Latn`) as the low-resource language in Part 2 or 3. The two parts look at the same question from two angles (subspaces of the hidden states versus individual neurons), but they train different technical skills. Choose according to what you want to learn:

- **Part 2** is about *analysing representations*. You will extract the hidden states of every layer and pool them over word pieces with masks; manipulate them with linear algebra (centring, standardisation, singular value decomposition, orthonormal bases, projections, random subspaces); train and evaluate linear probes with scikit-learn while keeping fitting and evaluation data strictly apart; and run cross-lingual retrieval with cosine similarity on parallel data. It has more questions and more interpretation to write.

- **Part 3** is about *intervening on a model*. You will use PyTorch forward hooks to read and to modify the internal activations of a transformer; accumulate statistics over tens of thousands of tokens; implement an entropy-based selection method from a published paper; compute a masked-language-modelling loss with reproducible masks; and design an ablation experiment with a matched random control. It needs about an hour of computation: start the ablations early.

- **Part 1**, if you do it, trains you to use a tokeniser (word pieces, word alignment, special tokens) and to compare languages on a parallel corpus with a rank correlation.

Each task is followed by numbered questions. The questions of the part(s) you choose (Part 2, Part 3 or both) and of the synthesis are not optional: they are what the report is assessed on. Tasks marked 'optional', and their questions, can be skipped. Answer the questions in your report, referenced by their number.

Good practice (applies throughout)

- **Every number needs a reference point.** Report chance levels, random baselines or ceilings next to your measurements.

- **Fit and evaluate on different data.** Estimate subspaces, means, probes and neuron statistics on `dev`; evaluate on `devtest`.

- **Look at every layer.** Most quantities in this practical are functions of the layer; plot them as such.

- **Fix your random seeds** and cache expensive computations (`np.save`).

Scientific background

From a vague hypothesis to testable predictions

The slogan 'languages are represented in disjoint subspaces' suffers from three problems.

'Disjoint' has no useful meaning for linear subspaces. Two linear subspaces always share the zero vector: the relevant notions are whether their intersection is trivial, or whether they are orthogonal. Moreover, in $\mathbb{R}^d$, two subspaces of dimensions k_A and k_B necessarily share a non-trivial subspace as soon as $k_A + k_B > d$, since $\dim(S_A \cap S_B) \geq k_A + k_B - d$. The representations of any language have non-zero variance in essentially every direction, so the subspace 'occupied' by a language is the whole space, unless we restrict ourselves to its *principal subspace*: the span of the k directions that carry most of its variance. The question then becomes one of degree: how much do the principal subspaces of two languages overlap?

Linear or affine? If every language were simply translated by its own offset vector ($x = z+\mu_L$, with z language-neutral), the point clouds of the different languages would be perfectly separable, although, once centred, they would span exactly the same subspace. Separability therefore says nothing about subspaces, and centring each language on its own mean is indispensable.

Representing the language vs representing content in a language. Encoding *which* language a sentence is written in is cheap: L languages with distinct means are linearly separable within an $(L-1)$-dimensional subspace. The interesting version of the hypothesis is about content: is the information that distinguishes one French sentence from another encoded in directions that are specific to French?

Operational version. Fix a layer. For a language L, let $X_L \in \mathbb{R}^{n\times d}$ be the matrix of sentence representations ($d = 768$), and $\bar{X}_L$ the same matrix centred on its own mean. Let $U_L \in \mathbb{R}^{d\times k}$ contain the k leading right singular vectors of $\bar{X}_L$ computed on the `dev` split (its principal subspace). The share of the variance of a matrix X captured by a subspace with orthonormal basis U is

$$c(X, U) = \frac{\|\bar{X}U\|_F^2}{\|\bar{X}\|_F^2} \in [0, 1]. \tag{1}$$

We compare three subspaces on the `devtest` representations X_B^{test} of a language B: the principal subspace U_A of another language A, the principal subspace U_B of B itself (estimated on different sentences, the *ceiling*), and a random k-dimensional subspace U_{rand} (the *floor*). The *normalised overlap* is

$$\omega_k(A \to B) = \frac{c(X_B^{\text{test}}, U_A) - c(X_B^{\text{test}}, U_{\text{rand}})}{c(X_B^{\text{test}}, U_B) - c(X_B^{\text{test}}, U_{\text{rand}})}. \tag{2}$$

The two hypotheses now make opposite, measurable predictions:

- H_{disjoint}: $\omega_k(A \to B) \approx 0$ — the principal subspace of A captures no more of B's variance than a random subspace;
- H_{shared}: $\omega_k(A \to B) \approx 1$ — the principal subspace of A captures B's variance as well as B's own; the identity of the language lives in a low-dimensional component, mainly the mean offset.

Everything in between is a matter of degree, and the answer may differ from one layer to the next.

The superposition hypothesis

Elhage et al. (2022) study toy models in which a network must store n sparse features in $m < n$ dimensions. They show that a network can represent more features than it has dimensions by assigning them to *almost* orthogonal directions: in high dimension, exponentially many vectors

can be pairwise almost orthogonal. The price is *interference*: when two superposed features are active at the same time, each leaks onto the other's direction. Interference is cheap when features are sparse (rarely active) and when a non-linearity can filter out small leaks. In their toy models, the way features are arranged also depends on how they co-occur: features that tend to be active together are preferably given orthogonal directions, whereas features that are rarely active together can share dimensions at little cost.

Why this matters here. Languages are almost mutually exclusive: apart from code-switching, a sentence is written in a single language, and most of mBERT's 104 languages are rare in its pre-training data. Language-specific features (a French determiner, a Russian case ending) are therefore ideal candidates for superposition. The superposition view predicts that the features of different languages share the *same* dimensions rather than occupying disjoint subspaces, even when the features themselves are language-specific. It also warns us that a high overlap ω would not prove that languages share features: language-specific features superposed in the same dimensions would produce the same principal subspaces. Besides, principal component analysis only sees high-variance directions, whereas each sparse feature carries little variance. To tell shared features apart from superposed language-specific ones, we will have to test whether a given direction *means the same thing* in different languages (Task 2.6).

Privileged basis. Elhage et al. (2022) also point out that the residual stream of a transformer has no privileged basis: nothing in the architecture singles out the coordinate axes, so only basis-independent quantities (subspaces, angles, projections) are meaningful there. The activations of the feed-forward layers, which go through an element-wise non-linearity, do have a privileged basis: there, it makes sense to talk about individual *neurons*. This is why Part 2 studies subspaces of the hidden states, while Part 3 studies feed-forward neurons. Note, however, that under superposition a neuron may be *polysemantic*, i.e. respond to several unrelated features. Note also that some element-wise operations (such as the gain of layer normalisation) can make certain coordinates special in practice, which leads us to rogue dimensions.

Rogue (outlier) dimensions

In BERT-like models, a handful of coordinates take very large values compared with all the others. Kovaleva et al. (2021) traced such outlier dimensions to the parameters of layer normalisation and showed that disabling them severely degrades the model. Timkey and Schijndel (2021) showed that these *rogue dimensions* dominate cosine similarity and Euclidean distance, so that similarity-based analyses mostly reflect a few coordinates. They contribute to the *anisotropy* of contextual representations: two random vectors have a high average cosine similarity (Ethayarajh 2019).

A diagnostic. Write $u = x/\|x\|$ for a normalised representation. For two representations x and y drawn independently, $\mathbb{E}[\cos(x, y)] = \sum_{i=1}^{d} \mathbb{E}[u_i]^2$. The term $CC_i = \mathbb{E}[u_i]^2$ is the contribution of

dimension i to the expected cosine similarity (Timkey and Schijndel 2021). If two or three terms account for most of the sum, cosine similarities are driven by those dimensions.

Why this matters here. Cross-lingual retrieval (Task 2.4) relies on cosine similarity, and captured variance (Task 2.5) weights each direction by its variance. Rogue dimensions can therefore dominate both measures. The standard remedy is to repeat the analyses after *standardising* each coordinate (subtracting its mean and dividing by its standard deviation) and to check whether the conclusions survive.

Practical set-up

The model: multilingual BERT

We use the cased multilingual BERT (Devlin et al. 2019), pinned to a specific revision of the Hugging Face checkpoint so that everyone uses exactly the same weights and tokeniser (Table 1).

Hub identifier	`google-bert/bert-base-multilingual-cased`
Revision (commit)	`3f076fdb1ab68d5b2880cb87a0886f315b8146f8`
Architecture	BERT-base encoder, post-layer-normalisation
Layers / hidden size / attention heads	12 / 768 / 12
Feed-forward size and activation	3072, GELU
Vocabulary	119,547 cased WordPiece units, shared by all languages
Maximum input length	512 positions
Parameters	178,566,653 (whole checkpoint, MLM head included)
Pre-training objectives	masked language modelling and next-sentence prediction
Pre-training data	Wikipedia in 104 languages

Table 1: The model used in this practical.

Pre-training data and vocabulary. mBERT was pre-trained on the Wikipedias of 104 languages, chosen among those with the largest Wikipedias. To limit the dominance of high-resource languages, the data were re-weighted by *exponential smoothing*: if p_L is the share of language L in the raw data, sentences were sampled with probability $q_L \propto p_L^S$ with $S = 0.7$, which under-samples English and over-samples languages such as Icelandic. The same smoothing was applied when building the WordPiece vocabulary. Before WordPiece is applied, spaces are inserted around every character of the CJK Unicode range, so that Chinese is effectively tokenised into characters; Japanese kana and Korean Hangul are not treated this way. The cased model does not lower-case text and does not strip accents.

Loading the model.

```python
import torch
from transformers import AutoTokenizer, AutoModelForMaskedLM

MODEL_ID = "google-bert/bert-base-multilingual-cased"
REVISION = "3f076fdb1ab68d5b2880cb87a0886f315b8146f8"   # pin the exact checkpoint

tok = AutoTokenizer.from_pretrained(MODEL_ID, revision=REVISION)
model = AutoModelForMaskedLM.from_pretrained(MODEL_ID, revision=REVISION).eval()
torch.set_grad_enabled(False)      # inference only: saves memory and time

print(len(tok))                    # should match config.vocab_size
print(model.bert.encoder.layer[0]) # attention, intermediate, output
```

A warning about unused weights (the next-sentence-prediction head) is expected. The parts of the model you will use are:

- `model.bert.embeddings`: token, position and segment embeddings, followed by layer normalisation;
- `model.bert.encoder.layer[l]` for $l = 0, \ldots, 11$, each made of `attention`, `intermediate` (a linear map $768 \to 3072$ followed by GELU: these are the 'neurons' of Part 3) and `output` (a linear map $3072 \to 768$, a residual connection and layer normalisation);
- `model.bert(..., output_hidden_states=True).hidden_states`: a tuple of 13 tensors of shape $(B, T, 768)$. Index 0 is the output of the embedding layer; index $l+1$ is the output of `encoder.layer[l]`. Because BERT normalises *after* each residual addition, these vectors are layer-normalised.

The data: FLORES+

FLORES+ (NLLB Team 2024) is a multilingual machine-translation benchmark maintained by the Open Language Data Initiative (version 4.6 at the time of writing). English sentences were sampled in equal amounts from Wikinews, Wikijunior and Wikivoyage and professionally translated into more than 200 language varieties. Each variety has a `dev` split (997 sentences) and a `devtest` split (1,012 sentences). Sentences with the same `id` in the same split are translations of each other: the corpus is *n-way parallel*. Besides `text` and `id`, each row has metadata fields, including `domain` (`wikinews`, `wikijunior` or `wikivoyage`) and `topic`.

Configurations are named `<ISO 639-3>_<ISO 15924>`, e.g. `fra_Latn`. Beware of a few codes that differ from FLORES-200: Mandarin is `cmn_Hans`, Swahili is `swh_Latn`, Standard Arabic is `arb_Arab`.

Access and licence. FLORES+ is distributed under CC BY-SA 4.0 and is intended for *evaluation only*. The repository is gated: log in on the Hugging Face website, accept the conditions on the dataset page, create an access token, and run `huggingface_hub.login()` once. Do not repost the files publicly. Local copies may be provided on the course page.

```python
import pandas as pd
from datasets import load_dataset

def load_flores(code, split):
    """One FLORES+ language variety as a DataFrame sorted by sentence id."""
    df = load_dataset("openlanguagedata/flores_plus", code, split=split).to_pandas()
    df["id"] = df["id"].astype(int)
    return df.sort_values("id").reset_index(drop=True)

# the six languages of Parts 2 and 3; add the codes of Table 2 for Part 1
LANGS = ["eng_Latn", "fra_Latn", "deu_Latn", "rus_Cyrl", "cmn_Hans", "yor_Latn"]
dev = {code: load_flores(code, "dev") for code in LANGS}
assert all((d["id"] == dev["eng_Latn"]["id"]).all() for d in dev.values())
```

Languages

Part 1 uses the 23 language varieties of Table 2. Parts 2 and 3 use six of them: `eng_Latn`, `fra_Latn`, `deu_Latn`, `rus_Cyrl`, `cmn_Hans`, plus one low-resource language that you will choose at the end of Part 1 (Yoruba, `yor_Latn`, if you skip Part 1).

Code	Language	Script	mBERT	Code	Language	Script	mBERT
`amh_Ethi`	Amharic	Ethiopic	no	`isl_Latn`	Icelandic	Latin	yes
`arb_Arab`	Arabic	Arabic	yes	`jpn_Jpan`	Japanese	Kanji+kana	yes
`ceb_Latn`	Cebuano	Latin	yes	`kor_Hang`	Korean	Hangul	yes
`cmn_Hans`	Mandarin	Han	yes	`mya_Mymr`	Burmese	Myanmar	yes
`deu_Latn`	German	Latin	yes	`rus_Cyrl`	Russian	Cyrillic	yes
`eng_Latn`	English	Latin	yes	`swh_Latn`	Swahili	Latin	yes
`fin_Latn`	Finnish	Latin	yes	`tgk_Cyrl`	Tajik	Cyrillic	yes
`fra_Latn`	French	Latin	yes	`tha_Thai`	Thai	Thai	yes
`hat_Latn`	Haitian Creole	Latin	yes	`war_Latn`	Waray	Latin	yes
`heb_Hebr`	Hebrew	Hebrew	yes	`wol_Latn`	Wolof	Latin	no
`hin_Deva`	Hindi	Devanagari	yes	`yor_Latn`	Yoruba	Latin	yes
				`zul_Latn`	Zulu	Latin	no

Table 2: Language varieties used in Part 1. The last column tells whether the language is among mBERT's pre-training languages.

Environment and computing budget

You need Python ≥ 3.10 with `torch`, `transformers`, `datasets`, `numpy`, `scipy`, `scikit-learn`, `pandas` and `matplotlib`. The code given in this document was tested with `torch` 2.5 and `transformers` 4.46 and 5.17. Everything runs on a CPU. Encoding the two splits of one language should take a few minutes on a recent laptop; with the suggested sizes, the deactivation experiments of Part 3 need about an hour of computation (much less on a GPU), so start them early and let them run while you answer the questions. Cache everything you compute (`np.save`, in `float16` if disk space is short) — but this is something you have, of course, mastered perfectly since last year.

Part 1 — Warm-up: how unequal is tokenisation? (optional)

This part is optional. If you skip it, use Yoruba (`yor_Latn`) as the low-resource language of Part 2 or 3.

Before looking at representations, look at what the model receives. mBERT splits text into WordPiece units drawn from a single vocabulary shared by 104 languages. Since this vocabulary was built from (smoothed) Wikipedia data, languages with little data received fewer dedicated units, and their words are likely to be cut into more pieces. The same sentence can therefore cost very different numbers of tokens in different languages (Petrov et al. 2023), and the quality of tokenisation is related to how well a multilingual model serves a language (Rust et al. 2021; Wu and Dredze 2020). The hypothesis of this part is simple: *the less pre-training data a language had, the more fragmented it is*.

Definitions. All quantities are computed on the `dev` split. Special tokens are left out: call the tokeniser with `add_special_tokens=False`. Let $n(s, L)$ be the number of word pieces of sentence s in language L.

- **Tokenisation premium** (Petrov et al. 2023): $\pi_L = \sum_s n(s, L) \,/\, \sum_s n(s, \text{eng})$, computed on parallel sentences.
- **Fertility** (Rust et al. 2021): φ_L = number of word pieces divided by number of words, where a 'word' is a unit produced by mBERT's own pre-tokeniser (splitting on whitespace and punctuation, and around CJK characters). The fast tokeniser gives, for each piece, the index of the word it comes from: `encoding.word_ids()`.
- **Proportion of continued words** (Rust et al. 2021): κ_L = proportion of words split into at least two pieces.
- **Unknown rate**: u_L = proportion of pieces equal to `[UNK]` (`tok.unk_token_id`).
- **Resource proxy**: W_L = number of articles in the Wikipedia of L, as listed on https://meta.wikimedia.org/wiki/List_of_Wikipedias (note the date you consulted it).

Task 1.1 — Look before you measure

Take the first sentence of `dev` in `eng_Latn`, `fra_Latn`, `fin_Latn`, `cmn_Hans`, `tha_Thai`, `wol_Latn` and `amh_Ethi`. Print its pieces (`tok.convert_ids_to_tokens`) together with `word_ids()`.

1. What does the prefix `##` mark, and why is it needed to reconstruct the original text?
2. How is the Chinese sentence split into words? Which preprocessing rule described in the set-up explains this, and what does it imply for the fertility of Chinese?
3. Compare Wolof and Amharic, two languages absent from mBERT's pre-training data. Which one produces `[UNK]` pieces, and is the cause the language or the script?

Task 1.2 — Measure fragmentation

4. Compute π_L, φ_L, κ_L and u_L for the 23 languages of Table 2, and collect W_L. Present them in a single table sorted by π_L.
5. Plot π_L against $\log_{10} W_L$, with a different marker for the languages absent from mBERT, and label each point with its code.
6. Compute Spearman's rank correlation between π_L and W_L over the languages seen by mBERT (`scipy.stats.spearmanr`).
7. Why do we compute the premium on parallel sentences, relative to English, rather than the average number of tokens per sentence of each language on its own corpus?
8. Why are the fertilities of Chinese, Japanese and Thai not comparable with those of the other languages?
9. Some languages combine a moderate premium with a high unknown rate. Which ones? Why is a low premium misleading in that case? For languages written without spaces between words, check `tok.backend_tokenizer.model.max_input_chars_per_word`: what happens to a 'word' longer than this limit?
10. Cebuano and Waray have some of the largest Wikipedias by number of articles. Where do they fall on your plot? Find out how most of their articles were created, and conclude on the validity of W_L as a proxy for pre-training data.
11. Without exponential smoothing ($S = 1$), would the relation between π_L and W_L be steeper or flatter? Justify with the formula for q_L.
12. Does your analysis support the hypothesis of this part? Give two reasons why fragmentation could affect the mean-pooled sentence representations used in Part 2.

Task 1.3 — Choose the low-resource language for Parts 2 and 3

Choose one language that is among mBERT's pre-training languages, has one of the highest premiums in your table, and has an unknown rate below 1%. Justify your choice in two sentences.

13. Why should languages with many `[UNK]` pieces be excluded from a geometric analysis of sentence representations?

Part 2 — Do languages share a subspace?

Do this part *or* Part 3 (or both, if you are feeling motivated).

This part implements the operational version of the hypothesis (see 'From a vague hypothesis to testable predictions'). We work with sentence representations: at each of the 13 layers, a sentence is represented by the mean of the hidden states of its word pieces, special tokens excluded. Everything is *fitted* on `dev` (997 sentences per language) and *evaluated* on `devtest` (1,012 sentences per language), for the six languages of Parts 2–3.

```python
import torch

def encode(sentences, max_length=256):
    enc = tok(sentences, padding=True, truncation=True, max_length=max_length,
              return_tensors="pt", return_special_tokens_mask=True)
    special = enc.pop("special_tokens_mask").bool()
    keep = enc["attention_mask"].bool() & ~special      # (B, T): real word pieces
    return enc, keep

def sentence_reps(sentences, batch_size=32):
    """Mean-pooled hidden states of all layers: array of shape (13, N, 768)."""
    chunks = []
    for i in range(0, len(sentences), batch_size):
        enc, keep = encode(sentences[i:i + batch_size])
        hs = model.bert(**enc, output_hidden_states=True).hidden_states
        H = torch.stack(hs)                             # (13, B, T, 768)
        w = keep.unsqueeze(-1).float()                  # (B, T, 1)
        chunks.append((H * w).sum(2) / w.sum(1))        # (13, B, 768)
    return torch.cat(chunks, dim=1).numpy()
```

Task 2.1 — Extract and inspect the representations

14. Compute and cache the representations of both splits for the six languages.

15. For each layer and language, compute the average norm of the vectors and the average cosine similarity between 1,000 random pairs of distinct sentences. Plot both against the layer.

16. What does `hidden_states[0]` contain? Why can the language of a sentence be identified at this layer without any contextual processing, at least for languages with a script of their own?

17. Why are `[CLS]` and `[SEP]` excluded from the average?

18. Everything is fitted on `dev` and evaluated on `devtest`. Why would it be a mistake to pool the vectors of all languages and split them at random into a fitting set and an evaluation set?

19. What do your average cosine similarities suggest about the geometry of the space?

Task 2.2 — Look for rogue dimensions (optional)

20. For each layer, normalise the vectors ($u = x/\|x\|$) and compute $CC_i = \bar{u}_i^2$ on the pooled `dev` representations of the six languages (see 'Rogue (outlier) dimensions'). Report the share of the three largest terms in $\sum_i CC_i$ and the indices of the corresponding dimensions.

21. Repeat for each language separately. Are the dominant dimensions the same?

22. Define a *standardised* version of the representations: subtract the mean and divide by the standard deviation of each coordinate, both estimated on the pooled `dev` representations of the six languages (one set of statistics per layer), and apply the same transformation to `devtest`. If you do this task, run Tasks 2.3 to 2.6 on both versions.

23. Show that $\sum_i CC_i$ estimates the expected cosine similarity between two independently drawn sentences.

24. Are the dominant dimensions shared across languages and layers?

25. Predict how dominant dimensions shared by all languages would bias (i) cosine-based retrieval, (ii) the overlap ω. You will check your predictions in Tasks 2.4 and 2.5.

26. Why do we pool the statistics over the six languages instead of standardising each language separately?

Task 2.3 — Can the language be decoded?

27. For each layer, train a logistic-regression classifier predicting the language of a sentence from its representation (fit on `dev`, evaluate on `devtest`). Use a `StandardScaler` followed by `LogisticRegression(max_iter=2000)` in a scikit-learn `Pipeline`.

28. Repeat after *per-language centring*: subtract from every vector the mean of its language, estimated on `dev`.

29. Plot accuracy against the layer for both conditions, with the chance level.

30. What is the chance level?

31. Accuracy is probably close to 100% at every layer. Why does this result say nothing about the disjoint-subspace hypothesis? How many dimensions does a linear classifier need to separate L classes with distinct means?

32. How much does accuracy drop after per-language centring? Explain why a linear classifier can hardly beat chance when the class means coincide, and what an accuracy remaining well above chance would reveal.

Task 2.4 — Is language identity just an offset?

For each language $X \neq$ English and each layer, perform cross-lingual retrieval on `devtest`: for each English sentence, find the most similar sentence (cosine similarity) among the 1,012 sentences in X, and compute precision at 1 (the proportion of English sentences whose nearest neighbour is their translation). Compare three variants: raw, per-language centred (means estimated on `dev`), and centred after standardisation (this last variant only if you did Task 2.2).

33. What is the precision at 1 of a random retrieval?

34. Interpret the effect of centring in terms of the distinction between linear and affine subspaces (see 'From a vague hypothesis to testable predictions').

35. At which layers is retrieval best? Compare with the literature (Pires, Schlinger and Garrette 2019; Libovický, Rosa and Fraser 2020).

36. How does your low-resource language compare with the others? If you did Part 1, relate your answer to it.

37. (Only if you did Task 2.2.) Does standardisation change the picture? Were your predictions of Task 2.2 correct?

Task 2.5 — Measure the overlap between principal subspaces

38. Implement $c(X, U)$ and $\omega_k(A \rightarrow B)$ (Eq. 2). Obtain principal subspaces with `np.linalg.svd(Xc, full_matrices=False)`, and a random orthonormal basis from the QR decomposition of a Gaussian matrix of shape (d, k).

39. For $k \in \{10, 50, 100\}$ and every layer, compute ω_k for all ordered pairs of distinct languages, on raw representations (and on standardised ones if you did Task 2.2).

40. Plot ω_k averaged over pairs against the layer, separating pairs that share a script from pairs that do not. At the layer of your choice, show the full 6×6 matrix as a heat map.

41. Show that, for any X, the expected value of $c(X, U_{\text{rand}})$ is k/d. (Hint: $\mathbb{E}[U_{\text{rand}} U_{\text{rand}}^{\top}] = \frac{k}{d} I_d$.) Check it numerically.

42. Why is the ceiling computed with U_B fitted on `dev` and evaluated on `devtest`? What would happen if you fitted and evaluated it on the same sentences?

43. How does ω_k vary with k? What happens when k approaches d? Why must k remain much smaller than the number of sentences used to estimate U_A?

44. Is ω symmetric? Interpret an asymmetry between English and your low-resource language.

45. At which layers is the overlap lowest? Is a low overlap at layer 0 between languages with different scripts evidence for H_{disjoint}?

46. (Only if you did Task 2.2.) Does standardisation change your conclusions? What does this tell you about rogue dimensions?

47. Suppose $\omega_k \approx 1$ in all middle layers. Does this establish that mBERT represents all languages with shared features? Use 'The superposition hypothesis'.

Task 2.6 — Same direction, same meaning?

We now test whether a direction that encodes some property in one language encodes the same property in another. We use the `domain` field of FLORES+ (three classes), which is identical for a sentence and its translations.

48. At two or three layers of your choice, train a logistic-regression probe predicting the domain on the `dev` representations of a language A. Evaluate it on the `devtest` representations of A (within-language accuracy) and of every other language B (transfer accuracy). Report the majority-class baseline.

49. Repeat with per-language centred representations.

50. Why are labels defined on the English source articles valid for every language?

51. Why are pairs of languages with different scripts the most informative here? (Think of what a probe could exploit when two languages share tokens, e.g. named entities or numbers.)

52. Why does centring matter for transfer?

53. Place your language pairs in the table below and say which scenario of 'The superposition hypothesis' your results support.

	transfer ≈ within-language	transfer ≈ baseline
high overlap ω	shared features	superposed specific features
low overlap ω	unlikely	disjoint subspaces

Part 3 — Language-specific neurons: re-implementing Tang et al. (2024)

Do this part *or* Part 2 (or both, if you want to make me happy). The code of this part uses the function `encode` (and, for Task 3.4, `sentence_reps`) given at the beginning of Part 2: copy it even if you do not do Part 2.

The original method. Tang et al. (2024) look for *neurons* rather than subspaces. A neuron is a column of the first matrix of a feed-forward block followed by the non-linearity; it is *activated* on a token when its activation value is positive. Their method, LAPE (*language activation probability entropy*), works as follows.

1. Feed the model monolingual texts (about 100 million Wikipedia tokens per language) and estimate, for neuron j of layer l and each language k, the activation probability $p^k_{l,j}$ (the proportion of tokens of language k on which the neuron is activated).

2. Normalise the vector $(p^1_{l,j}, \ldots, p^L_{l,j})$ so that it sums to one, giving $p'_{l,j}$, and compute its entropy $\mathrm{LAPE}_{l,j} = -\sum_k p'^k_{l,j} \log p'^k_{l,j}$. A low entropy means that the neuron fires mostly for one or two languages.

3. Keep the 1% of neurons with the lowest entropy. Assign a kept neuron to language k if $p^k_{l,j}$ exceeds a threshold τ, set to the 95th percentile of all activation probabilities. A neuron can thus be assigned to several languages.

4. Test the neurons causally: set the activations of the neurons of language k to zero and measure the increase in perplexity on every language. A good selection produces a large effect on the diagonal and small effects elsewhere.

They report such a diagonal pattern for several models, and they compare LAPE with random selection and with other selection criteria. For LLaMA-2 they find a strongly skewed U-shaped distribution over layers (many language-specific neurons in the first and last layers, few in between); for BLOOM, their per-layer counts are concentrated in the upper layers. They also show that activating or deactivating these neurons can steer the language in which the model answers.

Adapting the method to mBERT. mBERT has $12 \times 3072 = 36{,}864$ feed-forward neurons (the output of `encoder.layer[l].intermediate`). We estimate activation probabilities on the `dev` split (tens of thousands of tokens per language instead of 100 million), and we

replace perplexity, which is not defined for a masked language model, by the masked-language-modelling loss on `devtest` sentences. The code below counts activations and zeroes selected neurons with forward hooks.

```python
import torch.nn.functional as F

class FFNHooks:
    """Counts positive GELU activations and/or zeroes selected FFN neurons."""
    def __init__(self, model, ablate=None):
        self.layers = model.bert.encoder.layer
        self.counts = torch.zeros(len(self.layers), model.config.intermediate_size)
        self.n_tokens = 0
        self.keep = None             # (B, T) mask, set before each forward pass
        self.ablate = ablate or {}   # {layer index: LongTensor of neuron indices}

    def _hook(self, i):
        def hook(module, inputs, output):            # output: (B, T, 3072)
            if self.keep is not None:
                active = (output > 0) & self.keep.unsqueeze(-1)
                self.counts[i] += active.sum(dim=(0, 1))
            idx = self.ablate.get(i)
            if idx is not None and len(idx) > 0:
                output = output.clone()
                output[..., idx] = 0.0
                return output                        # replaces the module output
        return hook

    def __enter__(self):
        self.handles = [layer.intermediate.register_forward_hook(self._hook(i))
                        for i, layer in enumerate(self.layers)]
        return self

    def __exit__(self, *exc):
        for h in self.handles:
            h.remove()

def activation_probabilities(sentences, batch_size=32):
    """p[l, j] = P(neuron j of layer l is active | token of these sentences)."""
    with FFNHooks(model) as hooks:
        for i in range(0, len(sentences), batch_size):
            enc, keep = encode(sentences[i:i + batch_size])
            hooks.keep, hooks.n_tokens = keep, hooks.n_tokens + int(keep.sum())
            model.bert(**enc)
    return (hooks.counts / hooks.n_tokens).numpy()   # (12, 3072)

def masked_batches(sentences, p=0.15, seed=0, batch_size=32):
    """Masks drawn once: the same positions are masked in every condition."""
    g = torch.Generator().manual_seed(seed)
    batches = []
    for i in range(0, len(sentences), batch_size):
        enc, keep = encode(sentences[i:i + batch_size])
```

```
        m = (torch.rand(keep.shape, generator=g) < p) & keep
        labels = enc["input_ids"].masked_fill(~m, -100)
        enc["input_ids"] = enc["input_ids"].masked_fill(m, tok.mask_token_id)
        batches.append((enc, labels))
    return batches

def mlm_loss(batches, ablate=None):
    """Mean cross-entropy (nats) on masked positions, optionally with ablation."""
    total, n = 0.0, 0
    with FFNHooks(model, ablate=ablate):
        for enc, labels in batches:
            h = model.bert(**enc).last_hidden_state      # (B, T, 768)
            m = labels != -100                           # masked positions only
            logits = model.cls(h[m])                     # (n_masked, vocab)
            total += F.cross_entropy(logits, labels[m], reduction="sum").item()
            n += int(m.sum())
    return total / n
```

Start by computing, with `activation_probabilities`, the matrix $P^k \in [0,1]^{12\times 3072}$ on the `dev` sentences of each of the six languages. Everything that follows relies on it: cache it.

Task 3.1 — Inspect the activation probabilities (optional)

Plot, for each layer, the distribution of activation probabilities (pooled over languages), and count the neurons that are never, or almost always, active.

54. Show that, for GELU, a positive activation is equivalent to a positive pre-activation. Why is the sign of the activation only a crude summary of what a neuron does?

55. The standard error of an estimated probability is $\sqrt{p(1-p)/n}$. Compute it for your number of tokens n and compare with the setting of Tang et al. (2024). Is the precision sufficient for this analysis?

Task 3.2 — Select language-specific neurons with LAPE

56. Compute LAPE for every neuron, keep the lowest 1%, compute τ and assign the kept neurons to languages as described above.

57. Report the number of neurons assigned to each language, and to several languages.

58. Plot the number of selected neurons per layer, one curve per language.

59. Run the selection again without `deu_Latn`, and compute the Jaccard similarity between the sets of French neurons obtained with and without German.

60. Why must neurons that are never active in any language be excluded before computing LAPE?

61. Why normalise the activation probabilities before computing the entropy? What is the maximum value of LAPE with six languages?

62. Construct an example of a neuron with a very low LAPE that should *not* be called language-specific. Which step of the method excludes it?

63. Compare your layer distribution with that reported by Tang et al. (2024). An encoder trained with masked language modelling differs from a left-to-right decoder: could this explain a difference?

64. Which languages share neurons? Relate your answer to scripts (and to Part 1 if you did it).

65. What does the Jaccard similarity of question 59 tell you about the notion of a 'language-specific neuron'?

Task 3.3 — Deactivate the neurons

66. For each language, build `masked_batches` once from its first 300 `devtest` sentences, and compute the baseline loss.

67. For each language k, zero the neurons assigned to k and compute the loss on all six languages. Report the 6×6 matrix of loss increases (rows: deactivated language; columns: evaluated language).

68. Control: for each k, zero the same number of neurons *in each layer*, drawn at random among the neurons not assigned to any language. Average over three random draws.

69. Why are the masks drawn once and reused in every condition?

70. Why must the random control match the number of neurons layer by layer, rather than draw the same total number uniformly from the whole network?

71. Baseline losses differ across languages. Should you report absolute or relative increases? Justify.

72. Is your matrix diagonal? Which off-diagonal effects are largest, and why?

73. Does a strongly diagonal matrix show that the selected neurons are monosemantic 'French neurons'? Use the notion of polysemanticity.

Task 3.4 — Neurons and subspaces (optional)

Neuron j of `encoder.layer[l]` writes into the hidden states along the column w_j of `encoder.layer[l].output.dense.weight` (a 768×3072 matrix), i.e. into `hidden_states[l + 1]`. At that layer, let Q be an orthonormal basis of the span of the six centred language means (estimated on `dev`). For each selected neuron, compute $r_j = \|Q^\top w_j\|^2 / \|w_j\|^2$, and compare with neurons drawn at random in the same layers.

74. What value of r_j would you expect for a random direction, and why?

75. Do language-specific neurons write the 'language offset', i.e. the differences between the language means? What would it mean if they did not?

76. BERT adds the output of the feed-forward block to the residual stream and then applies layer normalisation. Why does this make the comparison only approximate?

Synthesis

Conclude your report with answers to the following questions (one page at most).

77. In two or three sentences, state what your results say about the hypothesis for mBERT, at which layers, and with which caveats.

78. Part 2 looks at subspaces, Part 3 at individual neurons, and the two may seem to give contradictory answers. Can a model both share a subspace across languages *and* have language-specific neurons? Use the notions of superposition and privileged basis.

79. What would you need to change to run this study on a decoder-only language model (which token representations, which loss, which layers)? Which result would lead you to change your conclusion?

80. Name the most important limitation of each part you did.

Going further. The most direct test of superposition would be to train a sparse autoencoder on mBERT's hidden states (Bricken et al. 2023) and count the features that are active in a single language versus several. This is feasible on a CPU with a few hundred thousand token vectors, but the quality of a sparse autoencoder trained on so little data is questionable: this is a topic for a Master's project rather than for a practical session.

References

Bricken, Trenton et al. (2023). 'Towards Monosemanticity: Decomposing Language Models With Dictionary Learning'. In: *Transformer Circuits Thread*.

Devlin, Jacob et al. (2019). 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding'. In: *Proceedings of NAACL-HLT 2019*.

Elhage, Nelson et al. (2022). 'Toy Models of Superposition'. In: *Transformer Circuits Thread*.

Ethayarajh, Kawin (2019). 'How Contextual are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings'. In: *Proceedings of EMNLP-IJCNLP 2019*.

Kovaleva, Olga et al. (2021). 'BERT Busters: Outlier Dimensions that Disrupt Transformers'. In: *Findings of ACL-IJCNLP 2021*.

Libovický, Jindich, Rudolf Rosa and Alexander Fraser (2020). 'On the Language Neutrality of Pre-trained Multilingual Representations'. In: *Findings of EMNLP 2020*.

NLLB Team (2024). 'Scaling Neural Machine Translation to 200 Languages'. In: *Nature* 630.

Petrov, Aleksandar et al. (2023). 'Language Model Tokenizers Introduce Unfairness Between Languages'. In: *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Pires, Telmo, Eva Schlinger and Dan Garrette (2019). 'How Multilingual is Multilingual BERT?' In: *Proceedings of ACL 2019*.

Rust, Phillip et al. (2021). 'How Good is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models'. In: *Proceedings of ACL-IJCNLP 2021*.

Tang, Tianyi et al. (2024). 'Language-Specific Neurons: The Key to Multilingual Capabilities in Large Language Models'. In: *Proceedings of ACL 2024*.

Timkey, William and Marten van Schijndel (2021). 'All Bark and No Bite: Rogue Dimensions in Transformer Language Models Obscure Representational Quality'. In: *Proceedings of EMNLP 2021*.

Wu, Shijie and Mark Dredze (2020). 'Are All Languages Created Equal in mBERT?' In: *Proceedings of the 5th Workshop on Representation Learning for NLP (RepL4NLP 2020)*.